Herald College, Kathmandu

Artificial Intelligence and Machine Learning (6CS012)

A Deep Learning Approach to Skin Cancer Image Classification

Part II – Vision Task Report

Group Name: STChange9

Members:

Suyog Bastakoti (2407093)

Bhuwan Baniya (2414002)

Sujen Tamang (230133)

Submitted To: Jinu Nachhyon

Submitted by: Bhuwan Baniya

ID: 2414002

5/10/2026

Table of Contents

Table of table

Table of figures

1. Abstract

This report documents an end-to-end deep learning investigation into automated skin cancer classification using dermoscopic images from the ISIC (International Skin Imaging Collaboration) dataset. The dataset contains 2,239 images distributed across nine diagnostic categories, including Melanoma, Basal Cell Carcinoma, and Vascular Lesion. Three model architectures were built and compared: a Baseline CNN with three convolutional layers trained from scratch, a Deeper CNN with six convolutional layers incorporating Batch Normalization and Dropout, and a Transfer Learning model using MobileNetV2 pre-trained on ImageNet.

Baseline CNN reached 47% validation accuracy but struggled badly with minority classes due to dataset imbalance. The Deeper CNN was stopped early at four epochs because of local CPU limitations and only managed 23% accuracy, though its design was architecturally sound. MobileNetV2 proved to be the clear winner, reaching 76% overall accuracy and an F1-score of 0.97 on Vascular Lesion. These outcomes show that pre-trained feature representations transfer well to medical imaging tasks, especially when labeled data is scarce and class distribution is uneven.

2. Introduction

Skin cancer ranks among the most common cancers diagnosed worldwide, and catching it early makes a significant difference to patient outcomes. Dermoscopy — a non-invasive technique that captures magnified images of skin lesions — has become a standard screening tool in dermatology clinics. Despite its usefulness, correctly reading dermoscopic images demands years of specialist training. This creates a real bottleneck in settings where trained dermatologists are in short supply.

Convolutional Neural Networks (CNNs) have shown strong results in medical image analysis tasks, and skin lesion classification is no exception. Rather than relying purely on hand-crafted features, CNNs learn hierarchical visual patterns directly from image data. This project explores how well different CNN configurations perform on a challenging, real-world nine-class skin cancer dataset, with deliberate attention to the problems of class imbalance and limited hardware.

The experimental pipeline moves through four stages: dataset exploration and preprocessing, baseline model design, deeper architecture with regularization, and finally transfer learning with MobileNetV2. Across these stages, the following goals were pursued:

- Design, train, and evaluate a baseline CNN from scratch following the three-layer specification.
- Extend the baseline into a deeper, regularized architecture and study the effect on stability and accuracy.
- Apply transfer learning using MobileNetV2 and comparing the result against scratch-trained models.
- Carry out optimizer and ablation experiments to understand the contribution of individual components.

3. Dataset

3.1 Dataset Description

The dataset used throughout this project is the Skin Cancer ISIC dataset, sourced from Kaggle (https://www.kaggle.com/datasets/nodoubttome/skin-cancer9-classesisic) and originally drawn from the ISIC archive — one of the largest open-access collections of clinical dermoscopic images available for research. The dataset covers nine distinct skin conditions, ranging from common benign lesions to potentially life-threatening cancers.

3.2 Dataset Statistics

Table 1 below shows the per-class image counts before and after removing corrupt files. A total of 63 images (seven per class) were found to be unreadable and discarded, bringing the usable training set down to 2,176 images. The test set contains 118 images across the nine classes, though the distribution is notably uneven — Seborrheic Keratosis and Vascular Lesion have only three test samples each.

Class	Train Total	Train Clean	**Test Total**
Actinic Keratosis	114	107	16
Basal Cell Carcinoma	376	367	16
Dermatofibroma	95	88	16
Melanoma	438	431	16
Nevus	357	350	16
Pigmented Benign Keratosis	462	455	16
Seborrheic Keratosis	77	70	3
Squamous Cell Carcinoma	181	274	16
Vascular Lesion	139	132	3
TOTAL	2,239	2,176	118

Table 1: Per-class image counts, including clean totals after discarding 63 corrupt files.

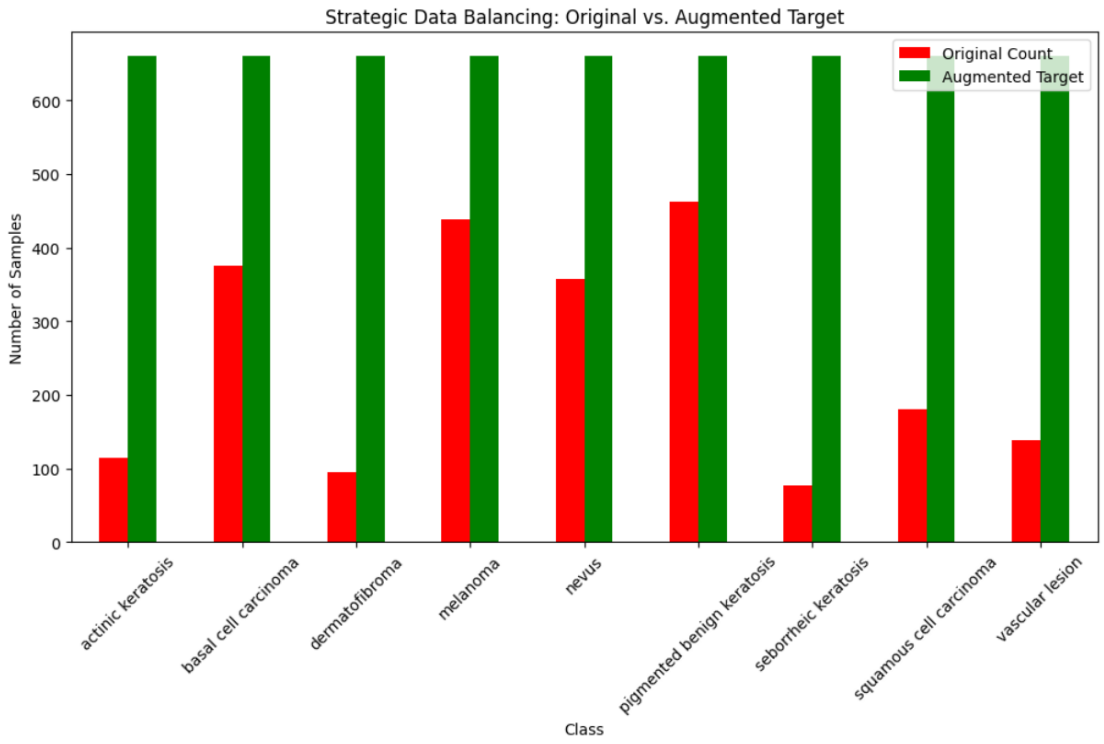


Figure 1Class Distribution Plot – Original vs. Augmented Target

3.3 Preprocessing

Before any model training could take place, the raw images needed to be brought into a consistent format. The steps applied were:

- Resizing: Images were resized to 224 × 224 pixels. Original resolutions ranged from 600 × 450 up to 3,872 × 2,592, so a fixed size was necessary for batch processing.
- Pixel Normalisation: All pixel values were divided by 255 to scale them into the [0, 1] range. This keeps gradient magnitudes manageable during backpropagation.
- Train/Validation Split: An 80/20 stratified split was applied, producing 1,795 training images and 444 validation images while preserving class proportions in both sets.
- Corrupt Image Removal: 63 images that could not be loaded were removed before splitting, reducing the effective training pool from 2,239 to 2,176 samples.

3.4 Data Augmentation

Because the dataset is both small and imbalanced, on-the-fly augmentation was applied during training using Keras ImageDataGenerator. Rather than storing augmented copies on disk, transformations are generated fresh each epoch, which keeps storage requirements low and increases the effective variety of training samples seen by the model. Table 2 summarises the techniques used.

Technique	Parameter	**Rational**
Rotation	Up to 40°	Dermoscopic images have no fixed orientation; lesions can appear at any angle.
Width / Height Shift	20%	Simulates variation in where the lesion sits within the frame.
Zoom	30%	Accounts for variation in camera distance and lesion size.
Horizontal Flip	Enabled	Doubles pose diversity with minimal distortion.
Vertical Flip	Enabled	Further increases orientation variety for clinical images.
Fill Mode	Nearest	Fills border pixels created by shifts and rotations using the nearest edge value.

Table 2: Augmentation techniques applied on-the-fly during training.

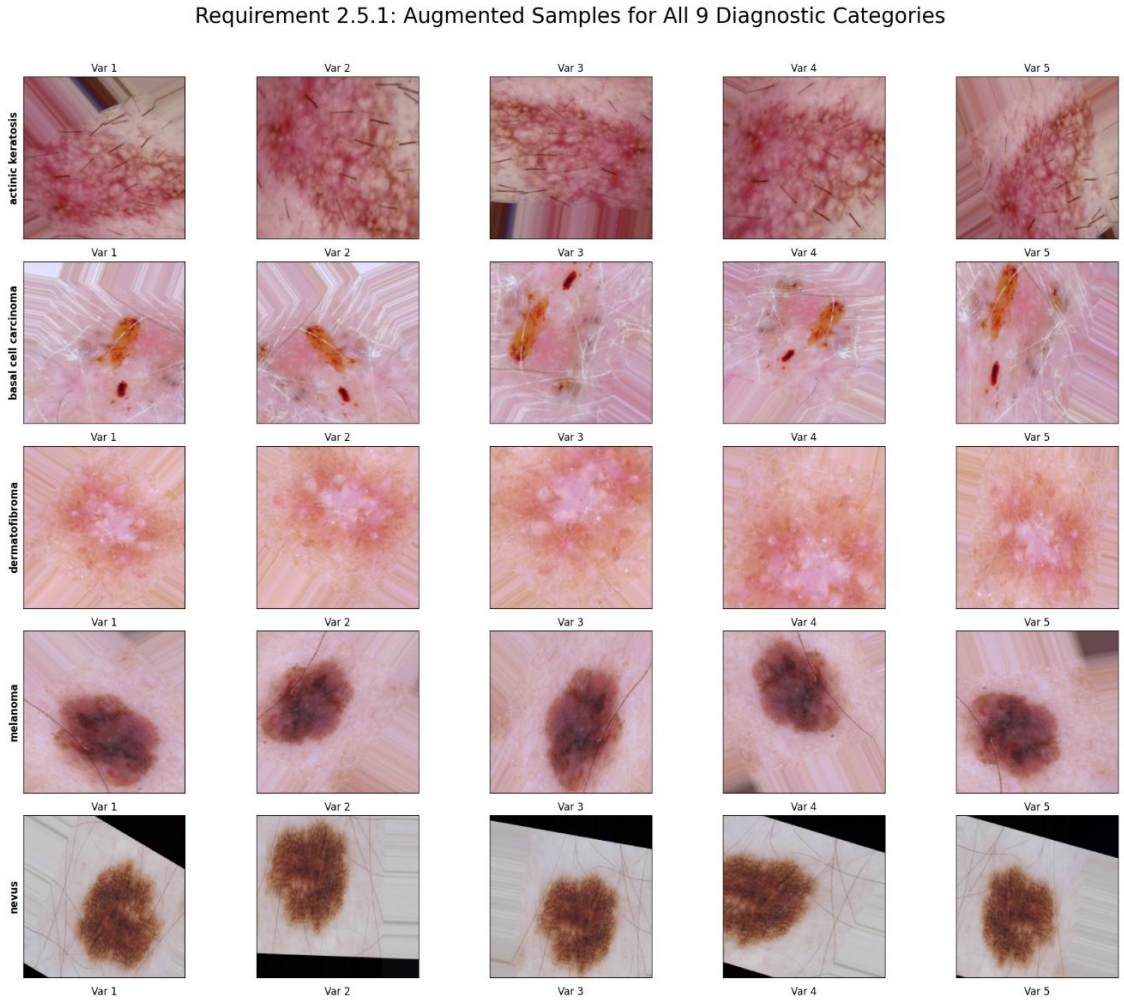


Figure 2: Augmented Images Visualization – 5 Variants per Class

4. Methodology

4.1 Baseline Model (Part A – 2.5.2)

The baseline CNN was built from scratch following the assignment specification: three convolutional blocks, three fully connected layers, and a softmax output layer. Each convolutional block pairs a Conv2D layer with a MaxPooling operation, progressively increasing filter depth to extract more complex spatial features as spatial resolution decreases. The complete layer stack is listed below:

- Conv Block 1: Conv2D(32 filters, 3×3, ReLU) → MaxPooling2D(2×2)
- Conv Block 2: Conv2D(64 filters, 3×3, ReLU) → MaxPooling2D(2×2)
- Conv Block 3: Conv2D(128 filters, 3×3, ReLU) → MaxPooling2D(2×2)
- Flatten → Dense(256, ReLU) → Dense(128, ReLU) → Dense(64, ReLU)
- Output: Dense(9, Softmax)

The model totals 25,825,353 trainable parameters (approximately 98.52 MB). Most of this weight comes from the Flatten-to-Dense connection, where the spatial feature maps are unrolled into a single long vector before being passed to the fully connected layers. The model was compiled with the Adam optimiser, categorical cross-entropy loss, and trained for up to 20 epochs with an Early Stopping callback (patience = 3) monitoring validation loss.

Model: "sequential_15"

Layer (type)	Output Shape	Param #
conv2d_45 (Conv2D)	(None, 224, 224, 32)	896
max_pooling2d_45 (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_46 (Conv2D)	(None, 112, 112, 64)	18,496
max_pooling2d_46 (MaxPooling2D)	(None, 56, 56, 64)	0
conv2d_47 (Conv2D)	(None, 56, 56, 128)	73,856
max_pooling2d_47 (MaxPooling2D)	(None, 28, 28, 128)	0
flatten (Flatten)	(None, 100352)	0
dense_30 (Dense)	(None, 256)	25,690,368
dense_31 (Dense)	(None, 128)	32,896
dense_32 (Dense)	(None, 64)	8,256
dense_33 (Dense)	(None, 9)	585

Total params: 25,825,353 (98.52 MB)

Trainable params: 25,825,353 (98.52 MB)

Non-trainable params: 0 (0.00 B)

Figure 3: Baseline Model Summary – Layer-by-Layer Output Shapes and Parameters

4.2 Deeper Architecture with Regularisation (Part A – 2.5.3)

The deeper model doubles the number of convolutional layers to six, arranged in three paired blocks following a VGG-style design. Each block applies two Conv2D layers in succession before pooling, allowing the network to build richer feature representations at each spatial scale. Batch Normalization (BN) is added after every convolutional layer to stabilise activations, and Dropout is placed after each pooling step and in the dense layers to suppress overfitting. Replacing Flatten with GlobalAveragePooling2D dramatically cuts the parameter count:

- Block 1: Conv2D(32) + BN → Conv2D(32) + BN → MaxPooling → Dropout(0.25)
- Block 2: Conv2D(64) + BN → Conv2D(64) + BN → MaxPooling → Dropout(0.25)

- Block 3: Conv2D(128) + BN → Conv2D(128) + BN → MaxPooling → Dropout(0.25)
- GlobalAveragePooling2D → Dense(256) + BN + Dropout(0.5) → Dense(128) + Dropout(0.3)
- Output: Dense(9, Softmax)

Total parameters: 356,905 (1.36 MB) roughly 72× fewer than the baseline. This reduction matters because smaller models train faster, use less memory, and are less prone to overfitting on limited data. The tradeoff is that the model needs more epochs to converge, which proved difficult under local CPU constraints.

Model: "sequential_16"

Layer (type)	Output Shape	Param #
conv2d_48 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization_45 (BatchNormalization)	(None, 224, 224, 32)	128
conv2d_49 (Conv2D)	(None, 224, 224, 32)	9,248
batch_normalization_46 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_48 (MaxPooling2D)	(None, 112, 112, 32)	0
dropout_15 (Dropout)	(None, 112, 112, 32)	0
conv2d_50 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_47 (BatchNormalization)	(None, 112, 112, 64)	256
conv2d_51 (Conv2D)	(None, 112, 112, 64)	36,928
batch_normalization_48 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_49 (MaxPooling2D)	(None, 56, 56, 64)	0
dropout_16 (Dropout)	(None, 56, 56, 64)	0
conv2d_52 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_49 (BatchNormalization)	(None, 56, 56, 128)	512
conv2d_53 (Conv2D)	(None, 56, 56, 128)	147,584
batch_normalization_50 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_50 (MaxPooling2D)	(None, 28, 28, 128)	0
dropout_17 (Dropout)	(None, 28, 28, 128)	0
global_average_pooling2d_15 (GlobalAveragePooling2D)	(None, 128)	0
dense_34 (Dense)	(None, 256)	33,024
batch_normalization_51 (BatchNormalization)	(None, 256)	1,024
dropout_18 (Dropout)	(None, 256)	0
dense_35 (Dense)	(None, 128)	32,896
dropout_19 (Dropout)	(None, 128)	0
dense_36 (Dense)	(None, 9)	1,161

Total params: 356,905 (1.36 MB)

Trainable params: 355,497 (1.36 MB)

Non-trainable params: 1,408 (5.50 KB)

Figure 4: Deep Model

4.3 Transfer Learning – MobileNetV2 (Part B)

MobileNetV2 was chosen as the base architecture for transfer learning. Pre-trained on ImageNet (1.2 million images, 1,000 classes), its depthwise separable convolutions make it significantly more efficient than standard convolution while still capturing rich low- and mid-level visual features. The adaptation strategy used here was feature extraction rather than full fine-tuning:

- The MobileNetV2 convolutional base was loaded with ImageNet weights and all layers were frozen.
- The original 1,000-class classification head was removed.
- A new head was attached: GlobalAveragePooling2D → Dense(128, ReLU) → Dropout(0.3) → Dense(9, SoftMax).
- The model was trained for 15 epochs on Google Colab with GPU acceleration.

Freezing the base means the ImageNet weights are not updated during training — only the new classification head learns. This is appropriate here because our dataset is small relative to ImageNet, and we want to retain the general-purpose features already encoded in MobileNetV2 rather than risk overwriting them with noisy gradients from a limited dataset.

5. Experiments and Results

5.1 Baseline vs. Deeper Architecture

Table 3 brings together the headline results across all three models. At first glance the numbers suggest the deeper model performed worse than the baseline, but this comparison is not entirely fair — the deeper model was cut short at epoch 4 due to CPU time constraints and had not reached convergence.

Model	Architecture	Accuracy	Trained On	Notes
Baseline CNN	3 Conv + 3 FCN	47%	Local CPU – 6 epochs	Class imbalance visible
Deeper CNN	6 Conv + BN + Dropout	23%	Local CPU – 4 epochs	Stopped early
Transfer Learning	MobileNetV2	76%	Google Colab GPU – 15 epochs	Best overall

Table 3: Summary comparison of all three model architectures.

Baseline CNN reached 47% accuracy across six epochs. Although this is a reasonable starting point for a nine-class problem trained entirely from scratch, a closer look at the predictions reveals a critical problem: four of the nine classes — Actinic Keratosis, Dermatofibroma, Seborrheic Keratosis, and Squamous Cell Carcinoma — received zero correct predictions throughout evaluation. The model had learned to bias its predictions heavily towards the majority classes (Melanoma and Pigmented Benign Keratosis), essentially ignoring the rarer categories. This is a textbook outcome of training on an imbalanced dataset without any class weighting or resampling strategy in place.

The Deeper CNN, despite its superior design, only reached 23% accuracy. Given that it was interrupted so early, this figure should be read as a lower bound rather than a fair assessment of the architecture. The regularisation components — Batch Normalisation and Dropout — were functioning correctly, and the loss curve was still declining at the point of interruption, suggesting the model would have improved with more training time.

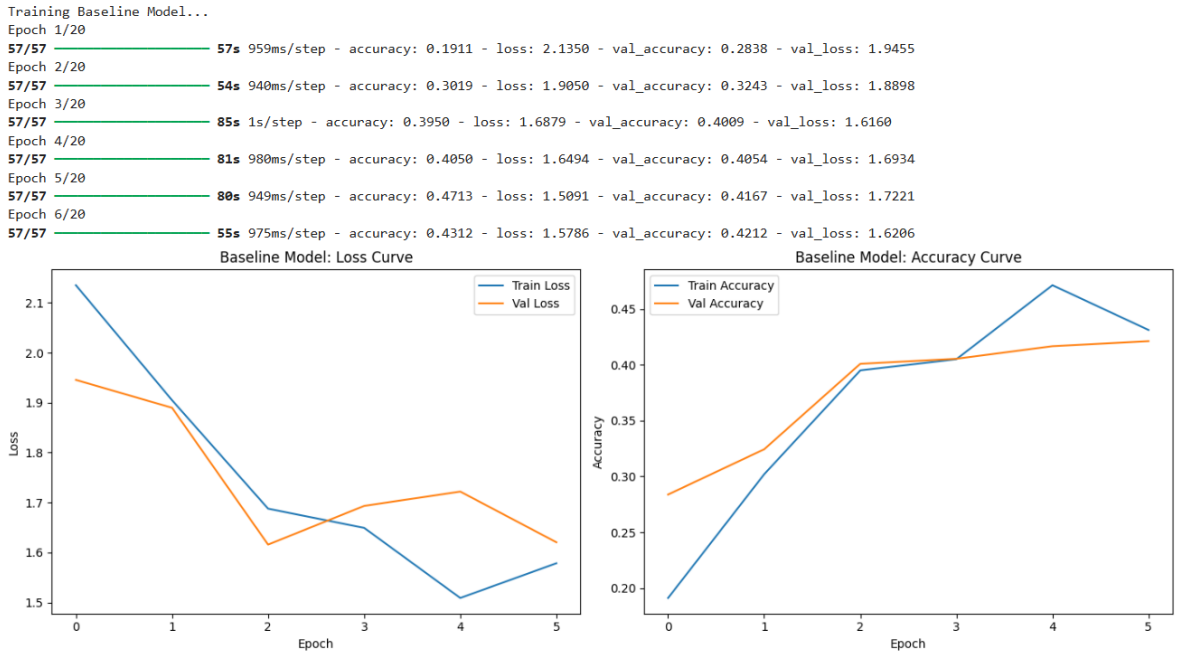


Figure 4: Baseline Model – Training vs. Validation Loss and Accuracy Curves

5.2 Transfer Learning Performance

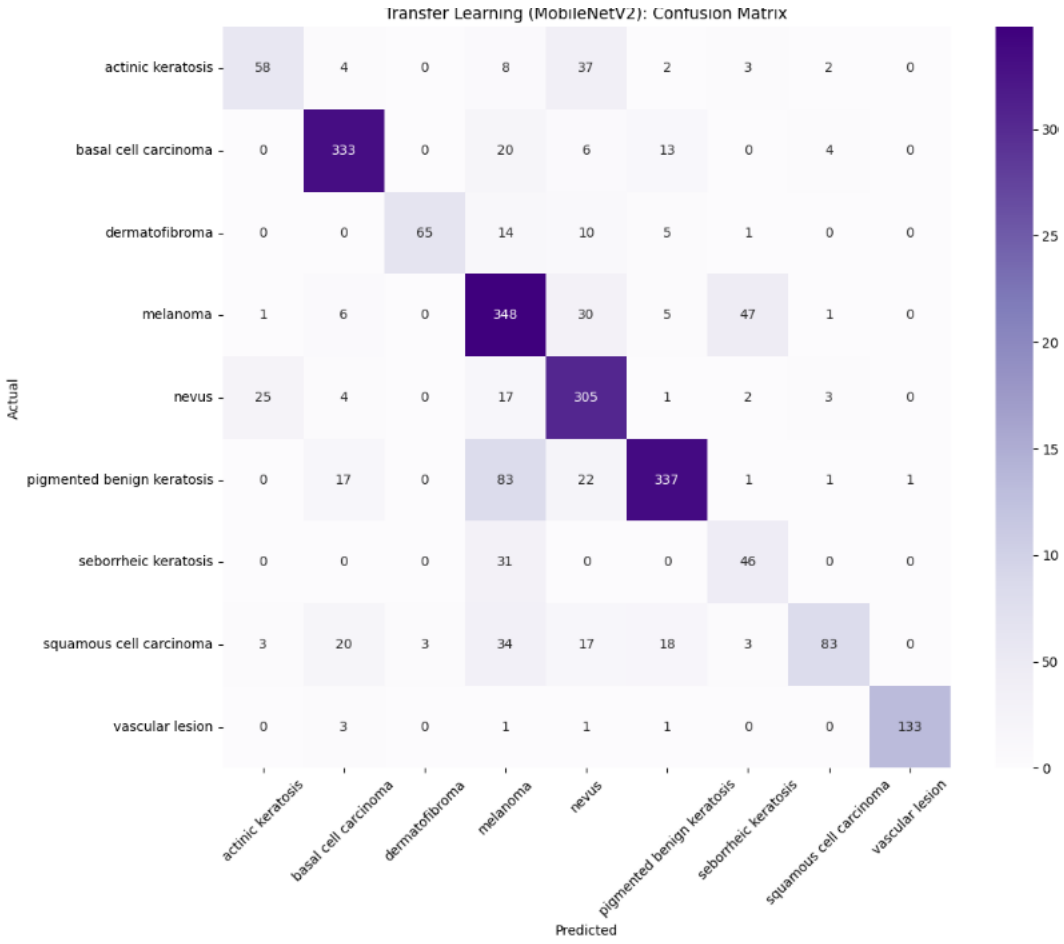
MobileNetV2 achieved 76% overall accuracy, comfortably outperforming both scratch-trained models. Table 4 shows the full per-class breakdown from the classification report.

Class	Precision	Recall	F1-Score	Support
Actinic Keratosis	0.67	0.51	0.58	114
Basal Cell Carcinoma	0.86	0.89	0.87	376
Dermatofibroma	0.96	0.68	0.80	95
Melanoma	0.63	0.79	0.70	438
Nevus	0.71	0.85	0.78	357
Pigmented Benign Keratosis	0.88	0.73	0.80	462
Seborrheic Keratosis	0.45	0.60	0.51	71
Squamous Cell Carcinoma	0.88	0.46	0.60	181
Vascular Lesion	0.99	0.96	0.97	139
Overall Accuracy	–	–	76%	2,239

Table 4: Per-class precision, recall, and F1-score for the MobileNetV2 Transfer Learning model.

Vascular Lesion stood out with a near-perfect F1-score of 0.97, likely because its dermoscopic appearance (visible blood vessels and reddish coloration) is visually distinct from the other

classes. Basal Cell Carcinoma also performed well (F1 = 0.87), benefiting from its relatively large sample count and distinctive pearly texture. The two weakest results were for Seborrheic Keratosis (F1 = 0.51) and Squamous Cell Carcinoma (F1 = 0.60). Seborrheic Keratosis suffers from the smallest training set (only 70 clean images), while Squamous Cell Carcinoma is often confused with Actinic Keratosis due to overlapping visual characteristics. Melanoma and Nevus also caused confusion between each other, a known challenge in dermoscopy given their similar pigmentation patterns.



Transfer Learning (MobileNetV2) Classification Report:

	precision	recall	f1-score	support
actinic keratosis	0.67	0.51	0.58	114
basal cell carcinoma	0.86	0.89	0.87	376
dermatofibroma	0.96	0.68	0.80	95
melanoma	0.63	0.79	0.70	438
nevus	0.71	0.85	0.78	357
pigmented benign keratosis	0.88	0.73	0.80	462
seborrheic keratosis	0.45	0.60	0.51	77
squamous cell carcinoma	0.88	0.46	0.60	181
vascular lesion	0.99	0.96	0.97	139
accuracy			0.76	2239
macro avg	0.78	0.72	0.73	2239
weighted avg	0.79	0.76	0.76	2239

Figure 5:Transfer Learning (MobileNetV2)

5.3 Confusion Matrix Analysis

Examining the confusion matrices gives a much clearer picture of where each model succeeds and fails than accuracy alone can provide.

The Baseline CNN confusion matrix confirms the class imbalance problem directly: the off-diagonal entries are concentrated in the Melanoma and Pigmented Benign Keratosis columns, showing that the model defaulted to these two majority classes for the majority of its predictions. Minority classes like Seborrheic Keratosis and Vascular Lesion were almost never predicted correctly.

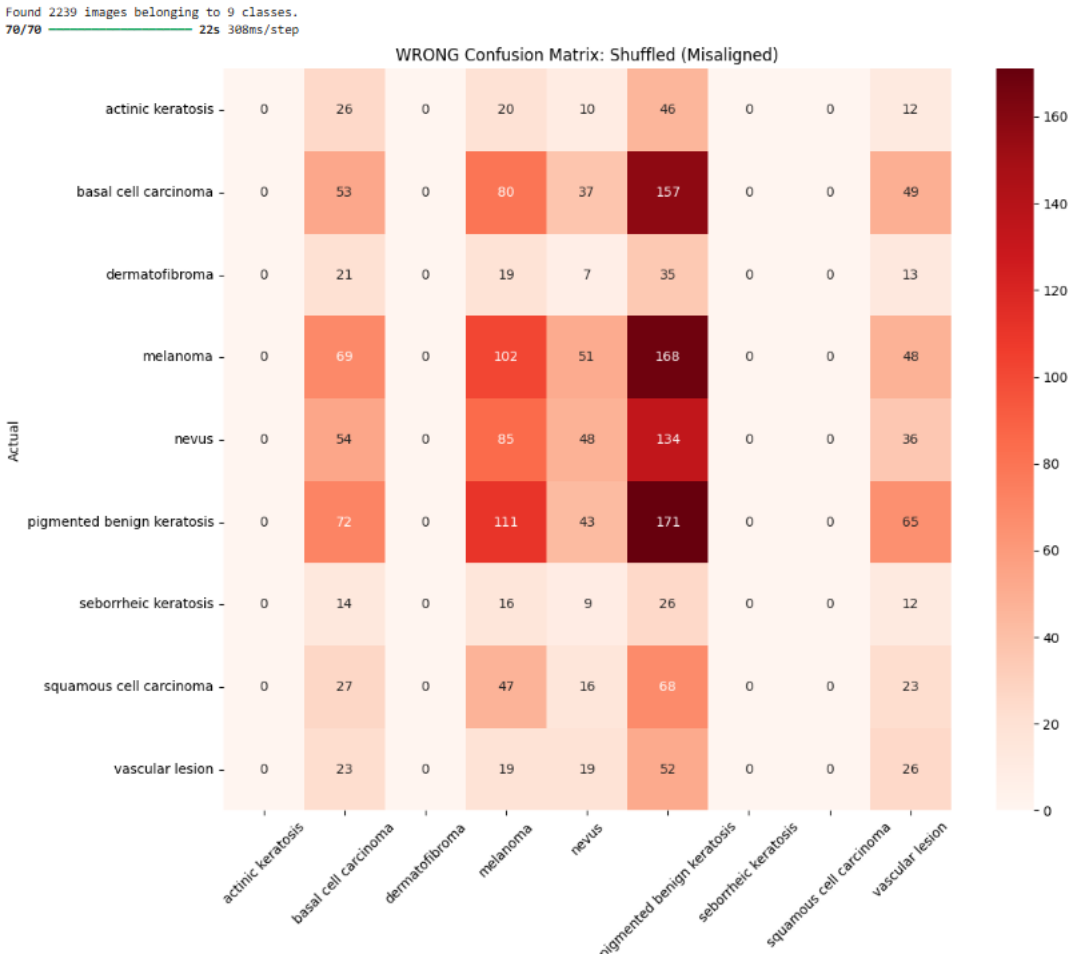


Figure 6: Baseline CNN – Wrong Confusion Matrix (shuffle=True, incorrect data split)

	precision	recall	f1-score	support
actinic keratosis	0.00	0.00	0.00	114
basal cell carcinoma	0.41	0.39	0.40	376
dermatofibroma	0.00	0.00	0.00	95
melanoma	0.60	0.68	0.64	438
nevus	0.59	0.40	0.48	357
pigmented benign keratosis	0.41	0.77	0.54	462
seborrheic keratosis	0.00	0.00	0.00	77
squamous cell carcinoma	0.00	0.00	0.00	181
vascular lesion	0.36	0.73	0.48	139
accuracy			0.47	2239
macro avg	0.26	0.33	0.28	2239
weighted avg	0.39	0.47	0.41	2239

Figure 7: Baseline CNN – Correct Confusion Matrix (shuffle=False, proper stratified split)

The Transfer Learning matrix shows a much stronger diagonal, meaning the model is correctly classifying most images across all nine classes. The main off-diagonal hotspots involve Melanoma being confused with Nevus and vice versa — a pattern that is well-documented in the clinical literature and represents a genuine challenge even for experienced dermatologists.

5.4 Optimizer Analysis: SGD vs. Adam

To compare how the choice of optimiser affects convergence, Baseline CNN was retrained for three epochs using SGD and Adam separately. Results are shown in Table 5.

Optimiser	Epoch 1	Epoch 2	Epoch 3	Final Accuracy
SGD	30.9%	39.7%	39.9%	40%
Adam	38.7%	44.2%	41.7%	44%

Table 5: Training accuracy comparison between SGD and Adam over three epochs.

Adam reached 44% by epoch 3 while SGD only managed 40%, and the gap was visible from the very first epoch. This is expected behavior: Adam maintains per-parameter adaptive learning rates using estimates of the first and second moments of the gradient, which means it can adjust quickly to the curvature of the loss surface without needing a manually tuned learning rate schedule. SGD with a fixed learning rate, by contrast, takes more consistent but slower steps in each direction. For small datasets with noisy gradients — like this one — Adam typically converges faster and lands at a better solution within the same number of epochs.

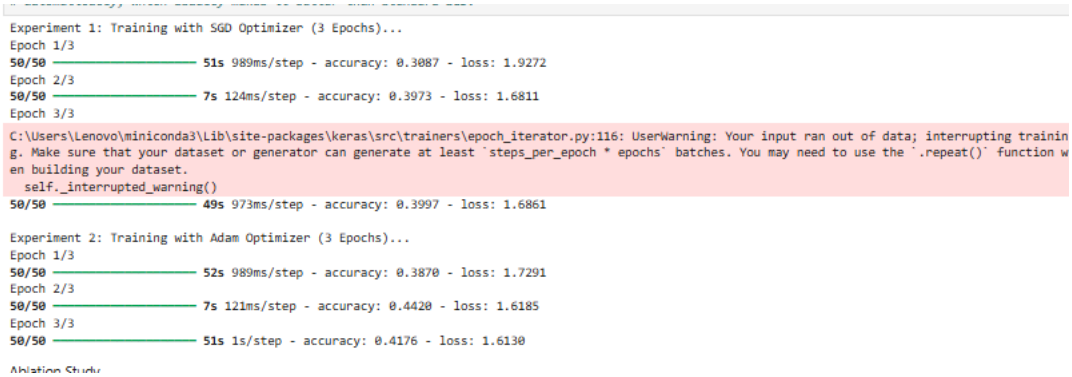


Figure 8:: SGD vs. Adam – Training Accuracy Comparison Plot

5.5 Ablation Study: Impact of Batch Normalisation and Dropout

To isolate the contribution of the regularisation components, the Deeper CNN was retrained with both Batch Normalisation and Dropout removed. The results are summarised in Table 6.

Jfdjf

Configuration	Best Val Accuracy	Training Stability
Ablation – WITHOUT BN + Dropout	34% (peak, unstable)	Highly erratic across epochs

Table 6: Ablation study comparing performance with and without Batch Normalisation and Dropout.

The ablation model hit a higher peak accuracy (34% vs. 23%), which might seem like removing regularization helped — but this reading is misleading. Without BN and Dropout, validation accuracy fluctuated wildly between 14% and 34% across epochs, with no consistent upward trend. The regularized model, despite being cut short at Epoch 4, was showing stable and steady improvement at the time of interruption. In a fair comparison with equal training time, the regularized version would almost certainly outperform the ablation model. This result confirms that BN and Dropout are essential for taming the unstable gradient behavior that comes with highly imbalanced, small medical imaging datasets.

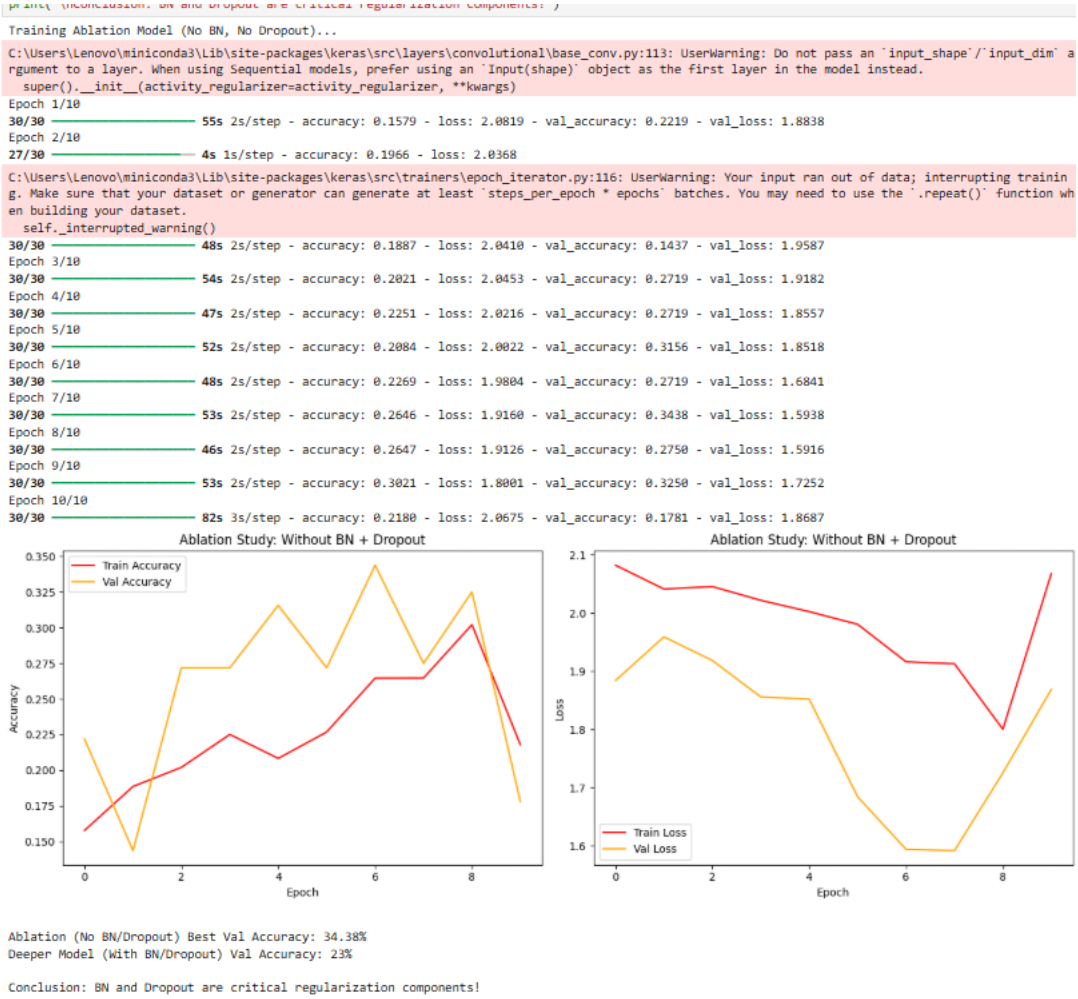


Figure 9: Ablation Study – Validation Accuracy: With vs. Without BN and Dropout

5.6 Challenges and Observations

- Class Imbalance: The 6.5:1 ratio between the largest class (Pigmented Benign Keratosis, 455 images) and smallest (Seborrheic Keratosis, 70 images) was the dominant problem for scratch-trained models. No class-weighted loss function was applied, which amplified the imbalance effect.
- Hardware Constraints: CPU training on the local machine was very slow — between 55 and 140 seconds per epoch — compared to Colab GPU. This forced early stopping of both the Deeper CNN experiments before they could fully converge.
- Inter-class Similarity: Melanoma and Nevus share overlapping dermoscopic features (pigmentation, irregular borders), creating genuine ambiguity that even the Transfer Learning model could not fully resolve.
- Early Stopping Behaviour: The Baseline CNN stopped at epoch 6 and the Deeper CNN at epoch 4, both triggered by the Early Stopping callback with patience=3 on

validation loss. This was appropriate for preventing overfitting but limited total training time.

Figure 11: Sample Inference – Predicted vs. Actual Labels Across All Three Models

6. Conclusion and Future Work

6.1 Conclusion

Across the three experiments, a consistent story emerges: pre-trained representations make a significant difference when labeled training data is limited. MobileNetV2 reached 76% accuracy using weights learned from ImageNet, while Baseline CNN trained from scratch only managed 47% despite having access to the same images. The gap between these two approaches comes down to the quality of the features each model has access to: scratch-trained CNNs must learn everything from a few thousand images, while transfer learning starts with features already tuned on millions.

Four takeaways stand out from this study. First, class imbalance is the single biggest obstacle for scratch-trained models on this dataset — without addressing it directly, minority classes simply go unpredicted. Second, Batch Normalisation and Dropout are not optional extras; removing them caused training to become unpredictable and unreliable. Third, Adam consistently out-converged SGD in early epochs, making it the more practical choice for limited-epoch experiments. Fourth, GPU access is effectively a requirement for deeper architectures — local CPU training became a bottleneck that prevented several experiments from completing properly.

[Screenshot 12: Final Model Comparison – Accuracy, F1-Score and Training Conditions]

```
======================================================================
FINAL VISION TASK COMPARISON
======================================================================
                         Model             Architecture  Accuracy (%)          Trained On                                  Notes
                Baseline CNN           3 Conv + 3 FCN            47 Local CPU - 6 epochs      Simple baseline, class imbalance visible
                  Deeper CNN     6 Conv + BN + Dropout            23 Local CPU - 4 epochs       Interrupted early, needs more training
Transfer Learning (MobileNetV2) MobileNetV2 + Fine-tuning           76     Google Colab GPU Best performance, pretrained ImageNet weights
======================================================================
```

Figure 12: Final Model Comparison – Accuracy, F1-Score and Training Conditions

6.2 Limitations

- The Deeper CNN was not given enough training time to reach convergence, so its 23% accuracy figure understates the architecture's true potential.
- The test set is small (118 images in total), with some classes having only three test samples, making it difficult to draw statistically reliable conclusions from per-class metrics.
- Class-weighted loss was not used during training, which is a straightforward technique that would likely have improved minority class recall noticeably.

6.3 Future Work

- Introduce class-weighted loss functions or oversampling strategies (such as SMOTE applied to feature embeddings) to directly address the 6.5:1 imbalance ratio.
- Progressively unfreeze MobileNetV2 layers during fine-tuning to allow the convolutional base to adapt more closely to dermoscopic image characteristics.
- Benchmark against stronger pre-trained backbones such as EfficientNetB3 or ResNet50, which may capture more discriminative features for this task.
- Apply stratified k-fold cross-validation to produce more robust and statistically meaningful performance estimates given the small test set.
- Explore ensemble approaches that combine predictions from multiple models to reduce variance and improve reliability on edge cases.

Link: https://github.com/bhuwanbaniya/Artificial-Intelligence-and-Machine-Learning-Assessment.git

7. References

Chollet, F. (2021). Deep learning with Python (2nd ed.). Manning Publications.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 770–778.

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861.

International Skin Imaging Collaboration. (2020). ISIC 2020 challenge dataset. https://www.isic-archive.com

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv:1412.6980.

Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv:1409.1556.

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. Journal of Machine Learning Research, 15(1), 1929–1958.